

LAPORAN PRAKTIKUM

STRUKTUR DAT

“Double Linked List”

disusun oleh:

Silva Nurifvan

2511533004

Dosen pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman. Tugas Individu ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, dengan Judul “Double Linked List”

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pengampu yang telah memberikan arahan sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami.

Padang, 14 Mei 2026

Silva Nurifvan

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum minggu ini, materi yang dipelajari adalah Double Linked List (DLL). Berbeda dengan Single Linked List yang hanya memiliki satu pointer (next), Double Linked List memiliki dua pointer yaitu next dan prev. Pointer next digunakan untuk menunjuk ke node berikutnya, sedangkan pointer prev digunakan untuk menunjuk ke node sebelumnya. Dengan adanya dua pointer ini, penelusuran data dapat dilakukan dari dua arah yaitu dari depan ke belakang maupun dari belakang ke depan.

Kelebihan ini membuat DLL lebih fleksibel dibandingkan SLL, meskipun membutuhkan memori yang lebih besar karena setiap node menyimpan dua referensi. Pada praktikum ini, saya ditugaskan untuk mengimplementasikan berbagai operasi dasar pada DLL seperti penambahan node (di awal, akhir, dan posisi tertentu), penghapusan node, serta penelusuran maju dan mundur. Pada praktikum kali ini, SLL dipelajari melalui berbagai operasi dasar seperti penambahan node di depan, di akhir, dan di posisi tertentu, pencarian data, penghapusan node, serta penelusuran (traversal) seluruh elemen.

1.2 Tujuan

Tujuan dari praktikum ini adalah memahami konsep Double Linked List dan perbedaannya dengan Single Linked List. Saya juga diharapkan mampu mengimplementasikan operasi penambahan node di berbagai posisi (insertBegin, insertEnd, insertAtPosition), operasi penghapusan node (deleteHead, deleteLast, deletePos), serta operasi penelusuran data dari dua arah (forward dan backward traversal).

1.3 Manfaat

Manfaat yang saya peroleh dari praktikum ini antara lain memahami cara kerja pointer ganda pada struktur data dinamis. DLL banyak digunakan dalam aplikasi nyata seperti fitur back dan forward pada browser, navigasi playlist musik, atau sistem undo-redo. Dengan memahami DLL, saya juga lebih siap untuk mempelajari struktur data yang lebih kompleks seperti Circular Double Linked List. Selain itu, praktikum ini melatih saya dalam melakukan debugging karena kesalahan pengaturan pointer pada DLL sering menyebabkan program error.

BAB II PEMBAHASAN

Double Linked List adalah struktur data linear yang terdiri dari kumpulan node. Setiap node pada DLL memiliki tiga komponen yaitu data (nilai yang disimpan), pointer next (menunjuk ke node setelahnya), dan pointer prev (menunjuk ke node sebelumnya). Node pertama disebut head, sedangkan node terakhir disebut tail. Pointer next pada tail bernilai null, dan pointer prev pada head juga bernilai null. Karena memiliki dua pointer, DLL memungkinkan penelusuran data dari dua arah, yaitu maju (dari head ke tail) dan mundur (dari tail ke head).

Kekurangan DLL dibandingkan SLL adalah penggunaan memori yang lebih besar dan waktu eksekusi yang sedikit lebih lama karena harus mengelola dua pointer setiap kali melakukan operasi insert atau delete.

2.2 Implementasi Pada Java

Berikut adalah penjelasan untuk masing-masing kode program Stack yang telah dipraktikkan.

2.2.1 NodeDLL_2511533004

```
1 package pekan6_2511533004;
2 public class NodeDLL_2511533004 {
3     // mendefinisikan kelas Node
4     int data_3004; // data
5     NodeDLL_2511533004 next_3004; // Pointer ke next node
6     NodeDLL_2511533004 prev_3004; // Pointer ke previous node
7
8     // konstruktor
9     public NodeDLL_2511533004(int data_3004) {
10         this.data_3004 = data_3004;
11         this.next_3004 = null;
12         this.prev_3004 = null;
13     }
14 }
```

Kelas ini berfungsi sebagai cetakan untuk membuat node-node dalam Double Linked List. Terdapat tiga atribut yaitu data_3004 bertipe int untuk menyimpan nilai, next_3004 bertipe NodeDLL_2511533004 untuk menunjuk ke node berikutnya, dan prev_3004 bertipe NodeDLL_2511533004 untuk menunjuk ke node sebelumnya. Ketiga atribut tersebut diberi akhiran 3020 sesuai aturan penamaan. Konstruktor NodeDLL_2511533004 (int data_3004) digunakan untuk menginisialisasi node baru dengan nilai tertentu, sekaligus mengatur next_3004 dan prev_3004 menjadi null.

2.2.2 InsertDLL_2511533004

```
1 package pekan6_2511533004;
2 public class InsertDLL_2511533004 {
3     // menambahkan node di awal DLL
4     static NodeDLL_2511533004 insertBegin(NodeDLL_2511533004 head_3004, int data_3004) {
5         // buat node baru
6         NodeDLL_2511533004 new_node_3004 = new NodeDLL_2511533004(data_3004);
7         // jadikan pointer pertama head
8         new_node_3004.next_3004 = head_3004;
9         // jadikan pointer kedua head ke new_node
10        if (head_3004 != null) {
11            head_3004.prev_3004 = new_node_3004;
12        }
13        return new_node_3004;
14    }
15
16    // fungsi menambahkan node di akhir
17    public static NodeDLL_2511533004 insertEnd(NodeDLL_2511533004 head_3004, int newData_3004) {
18        // buat node baru
19        NodeDLL_2511533004 newNode = new NodeDLL_2511533004(newData_3004);
20        // jika dll null jadikan head
21        if (head_3004 == null) {
22            head_3004 = newNode;
23        }
24        else {
25            NodeDLL_2511533004 curr = head_3004;
26            while (curr.next_3004 != null) {
27                curr = curr.next_3004;
28            }
29            curr.next_3004 = newNode;
30            newNode.prev_3004 = curr;
31        }
32        return head_3004;
33    }
34
35    // fungsi menambahkan node di posisi tertentu
36    public static NodeDLL_2511533004 insertAtPosition(NodeDLL_2511533004 head_3004, int pos_3004, int new_data_3004) {
37        // buat node baru
38        NodeDLL_2511533004 new_node_3004 = new NodeDLL_2511533004(new_data_3004);
39        if (pos_3004 == 1) {
40            new_node_3004.next_3004 = head_3004;
```

```
41            if (head_3004 != null) {
42                head_3004.prev_3004 = new_node_3004;
43            }
44            head_3004 = new_node_3004;
45            return head_3004;
46        }
47        NodeDLL_2511533004 curr = head_3004;
48        for (int i = 1; i < pos_3004 - 1 && curr != null; ++i) {
49            curr = curr.next_3004;
50        }
51        if (curr == null) {
52            System.out.println("Posisi tidak ada");
53            return head_3004;
54        }
55        new_node_3004.prev_3004 = curr;
56        new_node_3004.next_3004 = curr.next_3004;
57        curr.next_3004 = new_node_3004;
58        if (new_node_3004.next_3004 != null) {
59            new_node_3004.next_3004.prev_3004 = new_node_3004;
60        }
61        return head_3004;
62    }
63
64    public static void printList(NodeDLL_2511533004 head_3004) {
65        NodeDLL_2511533004 curr = head_3004;
66        while (curr != null) {
67            System.out.print(curr.data_3004 + " <-> ");
68            curr = curr.next_3004;
69        }
70        System.out.println();
71    }
72
73    public static void main(String[] args) {
74        // membuat dll 2 <-> 3 <-> 5
75        NodeDLL_2511533004 head_3004 = new NodeDLL_2511533004(2);
76        head_3004.next_3004 = new NodeDLL_2511533004(3);
77        head_3004.next_3004.prev_3004 = head_3004;
78        head_3004.next_3004.next_3004 = new NodeDLL_2511533004(5);
79        head_3004.next_3004.next_3004.prev_3004 = head_3004.next_3004;
```

```
76        head_3004.next_3004.next_3004 = new NodeDLL_2511533004(5);
77        head_3004.next_3004.next_3004.prev_3004 = head_3004.next_3004;
78
79        // cetak DLL awal
80        System.out.print("DLL Awal: ");
81        printList(head_3004);
82
83        // Langkah 1 di awal
84        System.out.print("simpul 1 ditambah di awal: ");
85        head_3004 = insertBegin(head_3004, 1);
86        printList(head_3004);
87
88        // Langkah 6 di akhir
89        System.out.print("simpul 6 ditambah di akhir: ");
90        int data_3004 = 6;
91        head_3004 = insertEnd(head_3004, data_3004);
92        printList(head_3004);
93
94        // menambahkan node 4 di posisi 4
95        System.out.print("tambah node 4 di posisi 4: ");
96        int data2_3004 = 4;
97        int pos_3004 = 4;
98        head_3004 = insertAtPosition(head_3004, pos_3004, data2_3004);
99        printList(head_3004);
100    }
101 }
```

Kelas ini berisi tiga method untuk menambahkan node ke dalam Double Linked List. Method insertBegin digunakan untuk menambahkan node di posisi paling depan. Langkahnya

adalah membuat node baru, mengarahkan pointer next node baru ke head yang lama, kemudian jika head tidak kosong, pointer prev dari head lama diarahkan ke node baru. Selanjutnya node baru dikembalikan sebagai head yang baru. Method insertEnd digunakan untuk menambahkan node di posisi paling belakang. Jika list kosong, node

baru langsung menjadi head. Jika tidak, program mencari node terakhir dengan perulangan while, lalu pointer next dari node terakhir diarahkan ke node baru, dan pointer prev dari node baru diarahkan ke node terakhir tersebut.

Method `insertAtPosition` digunakan untuk menambahkan node pada posisi tertentu. Jika posisi 1, maka method akan memanggil `insertBegin`. Jika posisi lebih dari 1, program menelusuri list sampai node sebelum posisi yang dituju, lalu pointer-pointernya diatur sedemikian rupa sehingga node baru tersisip di antara dua node yang sudah ada. Pada method `main`, dibuat DLL awal berisi 2,3,5. Selanjutnya dilakukan penambahan node 1 di awal, node 6 di akhir, dan node 4 di posisi 4. Hasil akhirnya DLL berisi 1,2,3,4,5,6.

```
<terminated> InsertDLL_2511533004 [Java Application] C:\Users\Lenovo\.p2\pool\plu
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambahkan di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambahkan di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->
```

2.2.3 PenelusuranDLL_2511533004

```
1 package pekan6_2511533004;
2 public class PenelusuranDLL_2511533004 {
3     //fungsi penelusuran maju
4     static void forwardTraversal_2511533004(NodeDLL_2511533004 head_3004) {
5         //mulai penelusuran dari head
6         NodeDLL_2511533004 curr_3004 = head_3004;
7         //lanjutkan sampai akhir
8         while (curr_3004 != null) {
9             //print data
10            System.out.print(curr_3004.data_3004 + " <-> ");
11            //pindah ke node berikutnya
12            curr_3004 = curr_3004.next_3004;
13        }
14        //print spasi
15        System.out.println();
16    }
17
18    //fungsi penelusuran mundur
19    static void backwardTraversal_2511533004(NodeDLL_2511533004 tail_3004) {
20        //mulai dari akhir
21        NodeDLL_2511533004 curr = tail_3004;
22        //lanjut sampai head
23        while (curr != null) {
24            //cetak data
25            System.out.print(curr.data_3004 + " <-> ");
26            //pindah ke node sebelumnya
27            curr = curr.prev_3004;
28        }
29        //cetak spasi
30        System.out.println();
31    }
32    public static void main(String[] args) {
33        //cetak DLL
34        NodeDLL_2511533004 head_3004 = new NodeDLL_2511533004(1);
35        NodeDLL_2511533004 second_3004 = new NodeDLL_2511533004(2);
36        NodeDLL_2511533004 third_3004 = new NodeDLL_2511533004(3);
37        head_3004.next_3004 = second_3004;
38
39        second_3004.prev_3004 = head_3004;
40        second_3004.next_3004 = third_3004;
41        third_3004.prev_3004 = second_3004;
42
43        System.out.println("Penelusuran maju:");
44        forwardTraversal_2511533004(head_3004);
45
46        System.out.println("Penelusuran mundur:");
47        backwardTraversal_2511533004(third_3004);
48    }
49 }
```

Kelas ini berisi dua method untuk melakukan penelusuran data pada Double Linked List. Method `forwardTraversal` digunakan untuk menampilkan data dari depan ke belakang. Penelusuran dimulai dari head, kemudian selama node tidak null, data

dicetak dan pointer berpindah ke node berikutnya. Method backwardTraversal digunakan untuk menampilkan data dari belakang ke depan.

Penelusuran dimulai dari tail (node terakhir), kemudian selama node tidak null, data dicetak dan pointer berpindah ke node sebelumnya. Pada method main, dibuat DLL sederhana berisi 1,2,3. Penelusuran maju menghasilkan 1,2,3, sedangkan penelusuran mundur menghasilkan 3,2,1.

```
<terminated> PenelusuranDLL
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

2.2.4 HapusDLL_2511533004

```
1 package pekan6_2511533004;
2
3 public class HapusDLL_2511533004 {
4
5     // Fungsi menghapus node paling awal (kepala)
6     public static NodeDLL_2511533004 delHead(NodeDLL_2511533004 head_3004) {
7         // Jika daftar kosong
8         if (head_3004 == null) {
9             return null;
10        }
11        // Geser kepala ke node berikutnya
12        head_3004 = head_3004.next_3004;
13        // Jika masih ada node, putuskan tautan mundur ke node lama
14        if (head_3004 != null) {
15            head_3004.prev_3004 = null;
16        }
17        return head_3004;
18    }
19
20    // Fungsi menghapus node paling akhir (ekor)
21    public static NodeDLL_2511533004 dellLast(NodeDLL_2511533004 head_3004) {
22        // Jika daftar kosong
23        if (head_3004 == null) {
24            return null;
25        }
26        // Jika hanya ada satu node
27        if (head_3004.next_3004 == null) {
28            return null;
29        }
30        // Telusuri sampai node paling akhir
31        NodeDLL_2511533004 curr = head_3004;
32        while (curr.next_3004 != null) {
33            curr = curr.next_3004;
34        }
35        // Putuskan tautan dari node sebelum ekor
36        curr.prev_3004.next_3004 = null;
37        return head_3004;
38    }
39 }
```

```

39
40 // Fungsi menghapus node pada posisi tertentu
41 public static NodeDLL_2511533004 delPos(NodeDLL_2511533004 head_3004, int pos_3004) {
42     // Jika daftar kosong
43     if (head_3004 == null) {
44         return head_3004;
45     }
46
47     NodeDLL_2511533004 curr = head_3004;
48
49     // Telusuri sampai mencapai posisi yang ingin dihapus
50     for (int i = 1; curr != null && i < pos_3004; ++i) {
51         curr = curr.next_3004;
52     }
53
54     // Jika posisi melebihi jumlah node / tidak ada
55     if (curr == null) {
56         System.out.println("Posisi tidak ditemukan!");
57         return head_3004;
58     }
59
60     // Hubungkan node sebelum dan sesudah node yang dihapus
61     if (curr.prev_3004 != null) {
62         curr.prev_3004.next_3004 = curr.next_3004;
63     }
64     if (curr.next_3004 != null) {
65         curr.next_3004.prev_3004 = curr.prev_3004;
66     }
67
68     // Jika node yang dihapus adalah node paling awal (head)
69     if (head_3004 == curr) {
70         head_3004 = curr.next_3004;
71     }
72
73     return head_3004;
74 }
75
76 // Fungsi mencetak seluruh isi DLL
77 public static void printList(NodeDLL_2511533004 head_3004) {
78     NodeDLL_2511533004 curr = head_3004;
79     while (curr != null) {
80         System.out.print(curr.data_3004 + " <-> ");
81         curr = curr.next_3004;
82     }
83     System.out.println();
84 }
85
86 // Fungsi Utama
87 public static void main(String[] args) {
88     // Membuat DLL awal: 1 <-> 2 <-> 3 <-> 4 <-> 5
89     NodeDLL_2511533004 head_3004 = new NodeDLL_2511533004(1);
90     head_3004.next_3004 = new NodeDLL_2511533004(2);
91     head_3004.next_3004.prev_3004 = head_3004;
92
93     head_3004.next_3004.next_3004 = new NodeDLL_2511533004(3);
94     head_3004.next_3004.next_3004.prev_3004 = head_3004.next_3004;
95
96     head_3004.next_3004.next_3004.next_3004 = new NodeDLL_2511533004(4);
97     head_3004.next_3004.next_3004.next_3004.prev_3004 = head_3004.next_3004.next_3004;
98
99     head_3004.next_3004.next_3004.next_3004.next_3004 = new NodeDLL_2511533004(5);
100    head_3004.next_3004.next_3004.next_3004.next_3004.prev_3004 = head_3004.next_3004.next_3004.next_3004;
101
102    // Menjalankan operasi penambahan
103    System.out.print("DLL Awal: ");
104    printList(head_3004);
105
106    System.out.print("Setelah head dihapus: ");
107    head_3004 = delHead(head_3004);
108    printList(head_3004);
109
110    System.out.print("Setelah node terakhir dihapus: ");
111    head_3004 = delLast(head_3004);
112    printList(head_3004);
113
114    System.out.print("menghapus node ke 2: ");
115    head_3004 = delPos(head_3004, 2);
116    printList(head_3004);
117 }
118 }

```

Kelas ini berisi tiga method untuk menghapus node dari Double Linked List. Method `delHead` digunakan untuk menghapus node paling depan. Caranya adalah dengan menggeser head ke node berikutnya, kemudian jika head baru tidak kosong, pointer `prev`-nya diatur menjadi null. Method `delLast` digunakan untuk menghapus node paling belakang. Program mencari node terlebih dahulu, kemudian pointer `next` dari node sebelumnya diatur menjadi null.

Method `delPos` digunakan untuk menghapus node pada posisi tertentu. Program menelusuri list sampai node yang akan dihapus ditemukan, lalu mengupdate pointer `next` dari node sebelumnya agar menunjuk ke node setelahnya, dan pointer `prev` dari node setelahnya agar menunjuk ke node sebelumnya. Jika node yang dihapus adalah

head, maka head digeser ke node berikutnya. Pada method main, dibuat DLL awal berisi 1,2,3,4,5. Selanjutnya dilakukan penghapusan head, penghapusan node terakhir, dan penghapusan node pada posisi 2. Hasil akhirnya DLL berisi 3,4.

```
<terminated> HapusDLL_2511533004 [Java Application] C:\Users\Lenov  
DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->  
Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->  
Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->  
menghapus node ke 2: 2 <-> 4 <->
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Double Linked List memiliki kelebihan dibandingkan Single Linked List yaitu kemampuan untuk melakukan penelusuran dari dua arah. Hal ini dimungkinkan karena setiap node pada DLL memiliki dua pointer yaitu next dan prev.

Operasi penambahan dan penghapusan node pada DLL memerlukan pengaturan kedua pointer dengan benar agar koneksi antar node tetap terjaga. Dari keempat program yang telah dibuat, operasi insert, delete, dan traversal pada DLL telah berhasil diimplementasikan sesuai dengan ketentuan yang diberikan.

3.2 Saran

Untuk pengembangan lebih lanjut, sebaiknya program dilengkapi dengan validasi input yang lebih baik, misalnya ketika posisi yang dimasukkan melebihi jumlah node, program dapat memberikan opsi untuk menambah node di akhir.

Selain itu, pada method `de1Last` di kelas `HapusDLL_2511533004`, sebaiknya ditambahkan penanganan khusus untuk list yang hanya berisi satu node. Secara umum, semua program sudah mengikuti aturan penamaan dengan akhiran NIM 3004.

3.3 Penutup

Demikian laporan praktikum Double Linked List ini saya buat. Laporan ini berisi penjelasan mengenai konsep DLL, implementasi kode, serta analisis dari keempat program yang telah dibuat. Saya menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat saya harapkan untuk perbaikan di masa mendatang.

TUGAS PRAKTIKUM

KELAS ADT

```
1 package pekan6_2511533004;
2
3 public class Lagu_2511533004 {
4
5     private String judul_3004;
6     private String penyanyi_3004;
7     private Lagu_2511533004 next_3004;
8     private Lagu_2511533004 prev_3004;
9
10    // Constructor
11    public Lagu_2511533004(String judul_3004, String penyanyi_3004) {
12        this.judul_3004 = judul_3004;
13        this.penyanyi_3004 = penyanyi_3004;
14        this.next_3004 = null;
15        this.prev_3004 = null;
16    }
17
18    // Getter
19    public String getJudul_3004() {
20        return judul_3004;
21    }
22
23    public String getPenyanyi_3004() {
24        return penyanyi_3004;
25    }
26
27    public Lagu_2511533004 getNext_3004() {
28        return next_3004;
29    }
30
31    public Lagu_2511533004 getPrev_3004() {
32        return prev_3004;
33    }
34
35    // Setter
36    public void setJudul_3004(String judul_3004) {
37        this.judul_3004 = judul_3004;
38    }
39
40    public void setPenyanyi_3004(String penyanyi_3004) {
41        this.penyanyi_3004 = penyanyi_3004;
42    }
43
44    public void setNext_3004(Lagu_2511533004 next_3004) {
45        this.next_3004 = next_3004;
46    }
47
48    public void setPrev_3004(Lagu_2511533004 prev_3004) {
49        this.prev_3004 = prev_3004;
50    }
51
52    public String toString() {
53        return judul_3004 + " - " + penyanyi_3004;
54    }
55 }
```

Kelas ini merepresentasikan satu buah lagu. Atributnya ada empat: judul_3004, penyanyi_3004, next_3004 (pointer ke lagu berikutnya), dan prev_3020 (pointer ke lagu sebelumnya). Semua atribut menggunakan akhiran 3004 sesuai aturan.

Kelas ini juga dilengkapi dengan constructor, getter, setter, dan method toString() untuk mencetak lagu dengan format "judul - penyanyi".

KELAS DRIVER

```
1 package pekan6_2511533004;
2
3 import java.util.Scanner;
4
5 public class Musik_2511533004 {
6     private Lagu_2511533004 head_3004;
7     private Lagu_2511533004 tail_3004;
8
9     // constructor
10    public Musik_2511533004() {
11        this.head_3004 = null;
12        this.tail_3004 = null;
13    }
14
15    public void tambahLagu_3004(String judul, String penyanyi) {
16        Lagu_2511533004 laguBaru = new Lagu_2511533004(judul, penyanyi);
17
18        if (head_3004 == null) {
19            head_3004 = laguBaru;
20            tail_3004 = laguBaru;
21        } else {
22            // sambung di akhir
23            tail_3004.setNext_3004(laguBaru);
24            laguBaru.setPrev_3004(tail_3004);
25            tail_3004 = laguBaru;
26        }
27        System.out.println("Lagu berhasil ditambahkan!");
28    }
29
30    // Metode hapus lagu pertama (diperbaiki)
31    public void hapusLaguPertama_3004() {
32        if (head_3004 == null) {
33            System.out.println("Playlist kosong! Tidak ada lagu yang dihapus.");
34            return;
35        }
36        if (head_3004 == tail_3004) { // Hanya ada 1 lagu
37            head_3004 = null;
38            tail_3004 = null;
39        } else {
40            head_3004 = head_3004.getNext_3004();
41            head_3004.setPrev_3004(null);
42        }
43        System.out.println("Lagu pertama berhasil dihapus!");
44    }
45
46    public void tampilkanSemuaLagu_3004() {
47        if (head_3004 == null) {
48            System.out.println("Playlist kosong! Tidak ada lagu yang dapat ditampilkan.");
49            return;
50        }
51        Lagu_2511533004 current = head_3004;
52        int urutan = 1;
53        while (current != null) {
54            System.out.println(urutan + ". " + current.toString());
55            current = current.getNext_3004();
56            urutan++;
57        }
58    }
59
60    public void tampilMaju_3004() {
61        if (head_3004 == null) {
62            System.out.println("Playlist kosong! Belum ada lagu.");
63            return;
64        }
65        System.out.println("=== PLAYLIST (Dari Awal ke Akhir) ===");
66        Lagu_2511533004 current = head_3004;
67        int urutan = 1;
68        while (current != null) {
69            System.out.println(urutan + ". " + current.toString());
70            current = current.getNext_3004();
71            urutan++;
72        }
73    }
74}
```

```

75 public void tampilMundur_3004() {
76     if (tail_3004 == null) {
77         System.out.println("Playlist kosong! Belum ada lagu.");
78         return;
79     }
80     System.out.println("=== PLAYLIST (Dari Akhir ke Awal) ===");
81     Lagu_2511533004 current = tail_3004;
82     int urutan = 1;
83     while (current != null) {
84         System.out.println(urutan + ". " + current.toString());
85         current = current.getPrev_3004();
86         urutan++;
87     }
88 }
89
90 // Metode cari lagu berdasarkan judul
91 public void cariLagu_3004(String judulCari) {
92     if (head_3004 == null) {
93         System.out.println("Playlist kosong!");
94         return;
95     }
96     Lagu_2511533004 current = head_3004;
97     boolean ditemukan = false;
98     while (current != null) {
99         if (current.getJudul_3004().equalsIgnoreCase(judulCari)) {
100             System.out.println("Lagu ditemukan: " + current.toString());
101             ditemukan = true;
102             break;
103         }
104         current = current.getNext_3004();
105     }
106     if (!ditemukan) {
107         System.out.println("Lagu dengan judul \"" + judulCari + "\" tidak ditemukan.");
108     }
109 }
110
111 // Metode utama untuk menjalankan program
112 public static void main(String[] args) {

```

```

113     Scanner input = new Scanner(System.in);
114     Musik_2511533004 playlist = new Musik_2511533004();
115     int pilihan;
116
117     System.out.println("=== Playlist Musik NIM: 2511533004 ===");
118     do {
119         System.out.println("1. Tambah Lagu");
120         System.out.println("2. Hapus Lagu Pertama");
121         System.out.println("3. Lihat Playlist (Maju)");
122         System.out.println("4. Lihat Playlist (Mundur)");
123         System.out.println("5. Cari Lagu");
124         System.out.println("6. Keluar");
125         System.out.print("Pilihan: ");
126         pilihan = input.nextInt();
127         input.nextLine(); // Memasukkan buffer input
128
129         switch (pilihan) {
130             case 1:
131                 System.out.print("Judul: ");
132                 String judul = input.nextLine();
133                 System.out.print("Penyanyi: ");
134                 String penyanyi = input.nextLine();
135                 playlist.tambahLagu_3004(judul, penyanyi);
136                 break;
137             case 2:
138                 playlist.hapusLaguPertama_3004();
139                 break;
140             case 3:
141                 playlist.tampilMaju_3004();
142                 break;
143             case 4:
144                 playlist.tampilMundur_3004();
145                 break;
146             case 5:
147                 System.out.print("Masukkan judul lagu yang dicari: ");
148                 String judulCari = input.nextLine();
149                 playlist.cariLagu_3004(judulCari);
150                 break;
151             case 6:

```

```

152                 System.out.println("Terima kasih telah menggunakan program ini...");
153                 break;
154             default:
155                 System.out.println("Pilihan tidak valid!");
156         }
157     } while (pilihan != 6);
158     input.close();
159 }
160 }

```

Kelas driver ini memiliki dua atribut utama: head_3004 (node pertama) dan tail_3004 (node terakhir). Method tambahLagu_3004() menambahkan lagu baru di akhir playlist dengan cara menghubungkan pointer next dari tail lama ke lagu baru, dan pointer prev dari lagu baru ke tail lama, lalu tail baru diupdate. Method hapusLaguAwal_3004() menghapus lagu pertama

dengan cara menggeser head_3004 ke node berikutnya, lalu prev dari head baru diatur menjadi null. Jika playlist kosong, program akan menampilkan pesan.

Method tampilMaju_3004() menelusuri dari head ke tail dengan menggunakan pointer next_3004. Method tampilMundur_3004() menelusuri dari tail ke head dengan menggunakan pointer prev_3004. Inilah keunggulan Double Linked List. Method cariLagu_3004() mencari lagu berdasarkan judul dengan perbandingan equalsIgnoreCase() sehingga tidak membedakan huruf besar dan kecil. Menu interaktif menggunakan perulangan do-while dengan 6 pilihan.

Output

```
=== Playlist Musik NIM: 2511533004 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan:
```